# Neural Network Training: Advanced Concepts

*(From Gradient-Based Learning to Maximum Likelihood Estimation)*
*(Ch 6)*

## 1. Gradient-Based Learning

Gradient-based learning refers to the process of training a neural network by iteratively adjusting its weights in the direction that reduces the loss, using gradients computed via backpropagation.

### Key Concepts:

- **Non-convex optimization**: Unlike linear models, neural networks have complex loss landscapes with many local minima and saddle points.
- **Initialization**: Weights are initialized to small random values to break symmetry and help gradient descent escape poor local minima.
- **Backpropagation**: Efficiently computes gradients for all weights using the chain rule.

### Author's Message:

*"Neural networks are trained using gradient descent, but the non-convex landscape makes initialization and optimization more challenging than in linear models. Backpropagation is the key to efficient gradient computation."*

### Example:

- **Task**: Train a neural network to classify handwritten digits (MNIST).
- **Initialization**: Weights are randomly initialized (e.g., using Xavier/Glorot initialization).
- **Training**: Gradient descent updates weights to minimize cross-entropy loss.

- **Outcome**: The network learns to classify digits, but the final weights depend on the initial random values.

## 2. Cost Functions

A cost function (or loss function) quantifies how well the neural network's predictions match the true labels. The goal of training is to minimize this function.

### Key Concepts:

- **Maximum Likelihood Estimation (MLE)**: Most cost functions are derived from MLE, which maximizes the probability of observing the true data under the model.
- **Saturation**: Some activation functions (e.g., sigmoid) can cause gradients to become very small, stalling learning.
- **Common Cost Functions**:
  - **Cross-Entropy**: For classification (derived from MLE for Bernoulli/Categorical distributions).
  - **Mean Squared Error (MSE)**: For regression (derived from MLE for Gaussian distribution).

### Author's Message:

*"Cost functions are not arbitrary; they are derived from probabilistic principles (MLE). Choose a cost function that matches your task and avoids gradient saturation."*

### Example:

- **Task**: Predict whether an email is spam (binary classification).
- **Cost Function**: Binary cross-entropy:

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

  where $y$ is the true label and $\hat{y}$ is the predicted probability.

- **Why?** This loss is the negative log-likelihood of the Bernoulli distribution, maximizing the likelihood of the observed labels.

## 3. Learning Conditional Distributions with Maximum Likelihood

Maximum Likelihood Estimation (MLE) is a method for estimating the parameters of a probabilistic model by maximizing the likelihood of the observed data. In neural networks, MLE is used to train the model to predict a conditional distribution $p(y|x;\theta)$.

### Key Concepts:

- **Conditional Distribution**: The model defines $p(y|x;\theta)$, where $\theta$ are the weights.
- **Likelihood**: The probability of observing the true data under the model's predicted distribution.
- **Log-Likelihood**: The log of the likelihood, used for numerical stability and to convert products into sums.
- **Negative Log-Likelihood**: The loss function minimized during training (equivalent to cross-entropy for classification).

### Author's Message:

*"Neural networks are probabilistic models. By maximizing the likelihood of the observed data, we train the model to predict distributions that match the true data-generating process. The choice of output activation and loss function implicitly defines this distribution."*

### Example:

- **Task**: Predict the probability of a coin landing heads.
- **Model**: A neural network with a sigmoid output (Bernoulli distribution).
- **Data**: 7 heads out of 10 flips.
- **Likelihood**: $L(\theta) = \theta^7(1-\theta)^3$.
- **Log-Likelihood**: $\log L(\theta) = 7\log(\theta) + 3\log(1-\theta)$.

- **Loss**: Negative log-likelihood: $-\log L(\theta)$.
- **Training**: The model adjusts $\theta$ to maximize $L(\theta)$.

## 4. Learning Conditional Statistics

Instead of learning the full conditional distribution $p(y|x)$, we sometimes only care about predicting a specific statistic (e.g., mean, variance) of $y$ given $x$.

### Key Concepts:

- **Conditional Mean**: For regression, we often predict the mean of $y$ given $x$.
- **Mean Squared Error (MSE)**: The loss function for predicting the mean, derived from MLE for a Gaussian distribution.
- **Functional View**: The neural network can approximate any function, and the cost function guides it to the desired statistic.

### Author's Message:

*"You don't always need the full distribution. For tasks like regression, predicting the mean (or other statistics) is sufficient. The cost function can be designed to directly optimize for these statistics."*

### Example:

- **Task**: Predict house prices (regression).
- **Model**: A neural network with a linear output (Gaussian distribution).
- **Loss**: MSE:

$$L = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

where $\hat{y}_i$ is the predicted mean.

- **Why?** Minimizing MSE is equivalent to finding the function that predicts the mean of $y$ given $x$.

## 5. Putting It All Together: How You Build a Neural Network

### Step-by-Step Process:

1. **Choose the Task**:
    - Classification (predict classes) or Regression (predict continuous values).
2. **Choose the Output Activation**:

| Task | Output Activation | Implicit Distribution |
|---|---|---|
| Binary Classification | Sigmoid | Bernoulli |
| Multi-class Classification | Softmax | Categorical |
| Regression | Linear | Gaussian |

3. **Choose the Loss Function**:

| Task | Loss Function |
|---|---|
| Binary Classification | Binary Cross-Entropy |
| Multi-class Classification | Categorical Cross-Entropy |
| Regression | Mean Squared Error (MSE) |

4. **Train the Model**:
    - The model learns weights that maximize the likelihood of the observed data (or minimize the loss).

### Example: Binary Classification in Code

```python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Step 1: Define the model (Bernoulli distribution)
model = Sequential([
    Dense(10, activation='relu', input_shape=(input_dim,)),
    Dense(1, activation='sigmoid')  # Sigmoid for Bernoulli
])

# Step 2: Choose the loss (negative log-likelihood of Bernoulli)
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',  # Cross-entropy for Bernoulli
    metrics=['accuracy']
)

# Step 3: Train the model (maximize likelihood)
model.fit(X_train, y_train, epochs=10)
```